

Le patron chaine de responsabilité

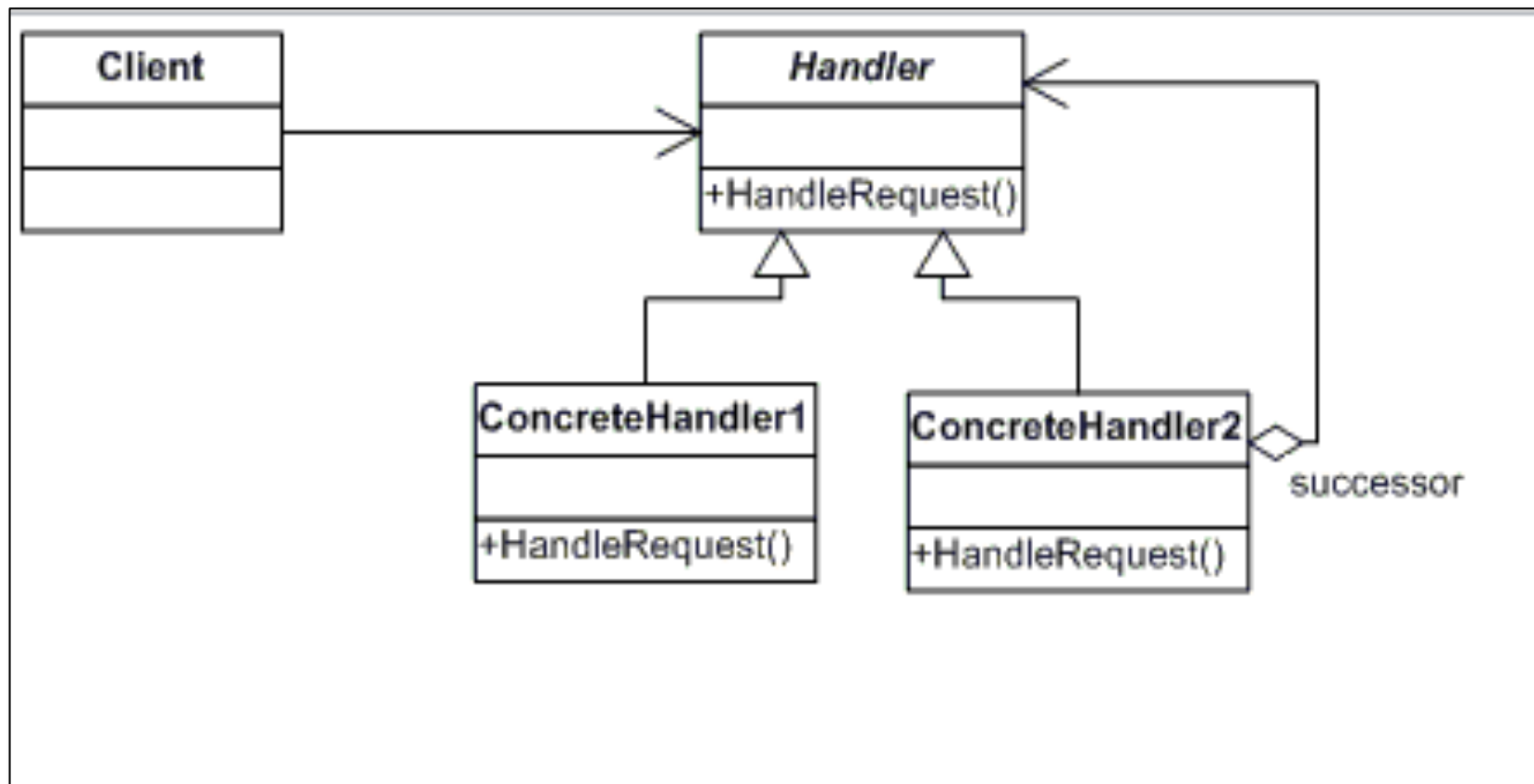
Chain of Responsibility

Préparé par MAA S.Yahiouche

But

- Utilisez le patron chaine de responsabilité lorsque vous voulez donner à plus d'un objet l'opportunité de gérer une requête

Diagramme UML



Participants

- Handler: définit l'interface de traitement de requêtes. Il peut implémenter le lien successeur.
- ConcreteHandler: traite la requête dont il est responsable, ou passe la requête à son successeur.
- Client: initialise une requête vers un ConcreteHandler dans la chaîne.

Chaine de responsabilité: Code structurel

classe Test

```
public class Test {  
public static void main( String arg[] ) {  
  
    Handler handler1 = new ConcreteHandler1();  
    Handler handler2 = new ConcreteHandler2();  
    Handler handler3 = new ConcreteHandler3();  
    Handler handler4 = new ConcreteHandler4();  
  
    handler1.setSuccessor( handler2 );  
    handler2.setSuccessor( handler3 );  
    handler3.setSuccessor( handler4 );  
  
    handler1.handleRequest(); }  
}
```

Chaine de responsabilité: Code structurel classe Handler

```
public class Handler {  
  private Handler successor;  
  
  public void setSuccessor( Handler successor ) {  
    this.successor = successor; }  
  
  public Handler getSuccessor() {  
    return successor; }  
  
  public void handleRequest() {  
    successor.handleRequest();  
  
  } }  
}
```

Chaine de responsabilité: Code structurel
classe ConcreteHandler1

```
public class ConcreteHandler1 extends  
    Handler {  
  
    public void handleRequest() {  
        if( /* nous pouvons gérer la demande == */  
            true ) { // Handle the request. }  
        else getSuccessor().handleRequest();  
  
    } }  
}
```

Exemple: essayer d'ouvrir un boulon à l'aide de plusieurs clé à pipe la classe abstraite Handler

```
public abstract class Handler{  
public static final int Clé1=12;  
public static final int Clé1=13;  
public static final int Clé1=14;
```

```
Private Handler successor;
```

```
public void setSuccessor(Handler successor)  
{this. successor=successor;}
```

```
public Handler getSuccessor(){return successor}
```

```
public abstract void ouvrirAvecCle(int DiamBou );
```

Exemple: essayer d'ouvrir un boulon à l'aide de
plusieurs clé à pipe
la classe Handler1 (1^{er} gestionnaire concret)

```
public class Handler1 extends Handler{

    public void ouvrirAvecCle(int DiamBou ){
    If ( DiamBou ==Cle1)
        System.out.println("la Clé "+Cle1+" a ouvert le boulon");
    else
        if (getSuccessor()!= null)
            getSuccessor(). ouvrirAvecCle( DiamBou );
    Else
        System.out.println("Ce boulon ne veut pas s'ouvrir avec mes clés");
    }

}
```

Exemple: essayer d'ouvrir un boulon à l'aide de
plusieurs clé à pipe
la classe Handler2 (2^{eme} gestionnaire concret)

```
public class Handler2 extends Handler{

    public void ouvrirAvecCle(int DiamBou ){
        If ( DiamBou == Cle2)
            System.out.println("la Clé "+Cle2+" a ouvert le boulon");
        else
            if (getSuccessor()!= null)
                getSuccessor(). ouvrirAvecCle( DiamBou );
        Else
            System.out.println("Ce boulon ne veut pas s'ouvrir avec mes clés");
        }

    }
```

Exemple: essayer d'ouvrir un boulon à l'aide de
plusieurs clé à pipe
la classe Handle3 (3^{eme} gestionnaire concret)

```
public class Handler3 extends Handler{

    public void ouvrirAvecCle(int DiamBou ){
        If ( DiamBou == Cle3)
            System.out.println("la Clé "+Cle3+" a ouvert le boulon");
        else
            if (getSuccessor()!= null)
                getSuccessor(). ouvrirAvecCle( DiamBou );
        Else
            System.out.println("Ce boulon ne veut pas s'ouvrir avec mes clés");
        }

    }
```

Exemple: essayer d'ouvrir un boulon à l'aide de plusieurs clé à pipe la classe Test

```
public class Test {  
public static void main( String arg[] ) {
```

```
    Handler handler1 = new Handler1();  
    Handler handler2 = new Handler2();  
    Handler handler3 = new Handler3();
```

```
    handler1.setSuccessor( handler2 );  
    handler2.setSuccessor( handler3 );
```

```
    handler1. ouvrirAvecCle(14);  
    handler1. ouvrirAvecCle(17); } }
```

```
*****
```

Console

```
*****
```

la Clé 14 a ouvert le boulon

Ce boulon ne veut pas s'ouvrir avec mes clés